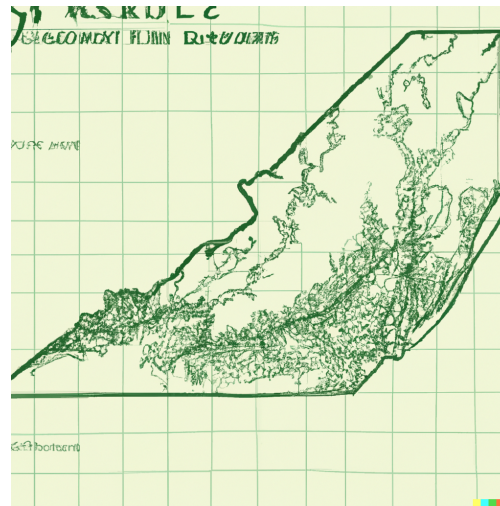


Import modules

```
In [99]: 1 import pandas as pd
2
3 from IPython.core.interactiveshell import InteractiveShell
4 InteractiveShell.ast_node_interactivity = "all"
5
6 from sklearn.tree import DecisionTreeClassifier as dt
7 from sklearn.ensemble import RandomForestClassifier as rf
8 from sklearn.model_selection import train_test_split
9 from sklearn.model_selection import GridSearchCV
```

Introduction



Map of New Maryland

You are the governor of the state of New Maryland. You signed a bill into law which requires that the court choose a risk assessment in order to aid judges in their pretrial detention decisions.

Some of your stakeholders want you to choose a risk assessment that targets felony crimes. These stakeholders think that, since felony crimes are more socially costly, and since the state budget has been cut, focusing on felony crimes is the most effective way to have the biggest impact on crime given limited resources. They want you to pair this algorithm with a tough-on-crime intervention that

detains high-risk individuals. The capacity for the program is **100**. The treatment effect of the intervention is **70%**.

Another set of stakeholders want you to choose a risk assessment that targets any crime, regardless of how severe. These stakeholders believe that preventing future crimes can only be achieved by signaling to current arrestees that no type of criminal offense will be treated leniently. These stakeholders also believe that the causes of crime are driven by mental health-related issues. They would like you to pair this algorithm with an intervention that provides online cognitive behavioral health therapy that meets once a month. The capacity for the program is **500**. The treatment effect of the intervention is **20%**.

Being a polymath, not only are you the governor of New Maryland, but you are also the state's Chief Data Scientist. In that role, you are going to build two prediction models.

- One will predict any re-arrest
- One will predict felony re-arrest

Then you will analyze the models and decide which is the most appropriate for your state.

First you will generate two features (to include with a set I have already created for you).

Then, for each outcome, you will run a decision tree model and a random forest model. You will choose the best model from these two (RF and decision trees) to then predict for the holdout.

Then you will evaluate each model + intervention.

Finally you will tell me which model you pick for your state.

Thresholds and treatment effects (1 point)

Create the following four variables:

- `threshold__any` : This should be set to the number of slots available in the online therapy intervention
- `threshold__felony` : This should be set to the number of slots available in the detention intervention
- `treatment_effect__online_therapy` : This should be set to the treatment effect of the online therapy intervention
- `treatment_effect__detention` : This should be set to the treatment effect of the detention intervention

```
In [101]: ▶ 1 threshold__any = 500  
          2 threshold__felony = 100  
          3 treatment_effect__online_therapy = .2  
          4 treatment_effect__detention = .7
```

Load Data

```
In [100]: 1 prediction_universe = pd.read_csv('https://zjelveh.github.io/files/prediction_universe_INST414.csv')
          2 arrests = pd.read_csv('https://zjelveh.github.io/files/arrests_INST414.csv')
          3 charges = pd.read_csv('https://zjelveh.github.io/files/charges_INST414.csv')
          4 prediction_universe.head()
          5
          6 arrests.head()
          7
          8 charges.head()
```

Out[100]:

	ArresteeID	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	group	outcome__any	outcome__felony
0	100684624.0	25.0	0.0	0.0	1.0	1.0	0	0
1	101611646.0	33.0	0.0	0.0	1.0	1.0	0	0
2	101790068.0	20.0	0.0	0.0	1.0	1.0	0	0
3	101696448.0	24.0	0.0	0.0	1.0	1.0	0	0
4	10179010.0	27.0	0.0	1.0	1.0	0.0	0	0

Out[100]:

	ArresteeID	arrests__arrest_year	ArrestNumber	arrests__weapon
0	102493798.0	2020	20-034161	Unarmed
1	101681693.0	2020	20-034161	Unarmed
2	102493798.0	2018	18-022775	Unarmed
3	101681693.0	2018	18-022775	Unarmed
4	102012407.0	2019	19-033083	Unarmed

Out[100]:

	ArrestNumber	ArChgNumID	Severity	NIBRS_Group	NIBRS_Crime_Category
0	18-031534	18-031534-01	M	B	DRIVING UNDER THE INFLUENCE
1	18-043911	18-043911-01	M	B	TRESPASS OF REAL PROPERTY
2	18-024199	18-024199-01	M	B	PUBLIC INTOXICATION
3	18-034153	18-034153-01	M	B	PUBLIC INTOXICATION
4	18-036191	18-036191-02	M	B	PUBLIC INTOXICATION

Column definitions:

- **prediction_universe:**
 - ArresteeID - unique ID for an arrestee
 - arrests__age - age of arrestee in 2018
 - arrests__total_YYYY - number of arrests by year YYYY (2016 to 2018)
 - group - an indicator of which group an arrestee belongs to (either 0 or 1). This variable is sensitive (akin to race/sex/gender/ethnicity/age/etc)
 - outcome__any - is equal to one if a person arrested in 2018 was re-arrested for any offense in 2019, 2020, or 2021
 - outcome__felony - is equal to one if a person arrested in 2018 was re-arrested for a felony offense in 2019, 2020, or 2021
- **arrests:**
 - ArresteeID - unique ID for an arrestee
 - arrests__arrest_year - year of arrest
 - ArrestNumber - unique ID for an arrest
 - arrests__weapon - type of weapon associated with arrest
- **charges:**
 - ArrestNumber - unique ID for an arrest
 - ArChgNumID - unique ID for an arrest charge
 - Severity - whether the charge was a felony or not
 - NIBRS_Group - NIBRS charge categorization I (A- Most severe, B, C-Least Severe)
 - NIBRS_Crime_Category - NIBRS charge categorization II (e.g. Public intoxication, DUI, Assault)

NOTE: NIBRS stands for [National Incident-Based Reporting System \(https://bjs.ojp.gov/national-incident-based-reporting-system-nibrs\)](https://bjs.ojp.gov/national-incident-based-reporting-system-nibrs), and is used by law-enforcement agencies across the country to report data to the federal government.

Generate more features

First feature (2 points)

Create a predictor called `charges__num_B__last_two_years` that counts the total number of arrests in 2017 and 2018 per ArresteeID where there was at least one charge that was NIBRS Group B. (You will only be adding one column). (1.5 points)

```
In [102]: 1 merged_arrests_charges = arrests.merge(charges, on="ArrestNumber")
          2 filtered_data = merged_arrests_charges[(merged_arrests_charges["arrests__arrest_year"] >= 2017) & (me
          3 charges__num_B__last_two_years = filtered_data.groupby("ArresteeID")["ArrestNumber"].count()
          4 charges__num_B__last_two_years
```

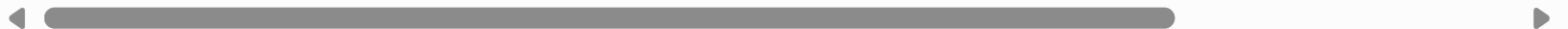
```
Out[102]: ArresteeID
2549297.0      1
2785936.0      2
2856607.0      4
2921408.0      1
2933651.0      3
..
102693701.0    2
102720125.0    1
102720805.0    2
102724697.0    2
102730586.0    2
Name: ArrestNumber, Length: 5753, dtype: int64
```

```
In [103]: 1 prediction_universe = pd.merge(prediction_universe, charges__num_B__last_two_years, on= 'ArresteeID',
2 prediction_universe.fillna('null', inplace=True)
3 prediction_universe
```

Out[103]:

	ArresteeID	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	group	outcome__any	outcome__felo
0	100684624.0	25.0	0.0	0.0	1.0	1.0	0	
1	101611646.0	33.0	0.0	0.0	1.0	1.0	0	
2	101790068.0	20.0	0.0	0.0	1.0	1.0	0	
3	101696448.0	24.0	0.0	0.0	1.0	1.0	0	
4	10179010.0	27.0	0.0	1.0	1.0	0.0	0	
...	...	...	...	...	...	...	...	...
7758	102088907.0	35.0	0.0	1.0	1.0	1.0	1	
7759	101649026.0	28.0	0.0	0.0	1.0	0.0	1	
7760	101865070.0	33.0	0.0	0.0	1.0	0.0	1	
7761	10109481.0	35.0	0.0	0.0	1.0	0.0	1	
7762	25073448.0	44.0	0.0	0.0	1.0	1.0	1	

7763 rows × 10 columns



Second feature (2 points)

Create a predictor called `arrests__weapon_used` that counts the number of arrests in 2016, 2017, and 2018 where the arrest weapon was NOT "Unarmed" or "None". (You will only be adding one column). (1.5 points)

Merge this feature in with `prediction_universe`. Fill any null values with zero. Make sure that the number of rows in `prediction_universe` does not change. (0.5 points)

```
In [104]: ▶ 1 f_e = arrests[arrests['arrests__arrest_year'].isin([2016,2017,2018]) & ((arrests['arrests__weapon']
2 arrests__weapon_used = f_e.groupby("ArresteeID")["arrests__arrest_year"].count()
3 arrests__weapon_used
4
5 prediction_universe = pd.merge(prediction_universe, arrests__weapon_used, on= 'ArresteeID', how= 'left')
6 prediction_universe.fillna('null', inplace=True)
```

```
Out[104]: ArresteeID
2549297.0      1
2785936.0      1
2856607.0      4
2921408.0      1
2933651.0      2
..
102693701.0    3
102720125.0    1
102720805.0    3
102724697.0    2
102730586.0    2
Name: arrests__arrest_year, Length: 7763, dtype: int64
```

```
In [105]: 1 prediction_universe.rename(columns={"ArrestNumber": "charges__num_B__last_two_years"}, inplace=True)
          2 prediction_universe.rename(columns={"arrests__arrest_year": "arrests__weapon_used"}, inplace=True)
          3 prediction_universe
```

Out[105]:

	ArresteeID	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	group	outcome__any	outcome__felo
0	100684624.0	25.0	0.0	0.0	1.0	1.0	0	
1	101611646.0	33.0	0.0	0.0	1.0	1.0	0	
2	101790068.0	20.0	0.0	0.0	1.0	1.0	0	
3	101696448.0	24.0	0.0	0.0	1.0	1.0	0	
4	10179010.0	27.0	0.0	1.0	1.0	0.0	0	
...	...	...	...	...	...	...	...	
7758	102088907.0	35.0	0.0	1.0	1.0	1.0	1	
7759	101649026.0	28.0	0.0	0.0	1.0	0.0	1	
7760	101865070.0	33.0	0.0	0.0	1.0	0.0	1	
7761	10109481.0	35.0	0.0	0.0	1.0	0.0	1	
7762	25073448.0	44.0	0.0	0.0	1.0	1.0	1	

7763 rows × 11 columns

Prepare data

Part I (0.5 points)

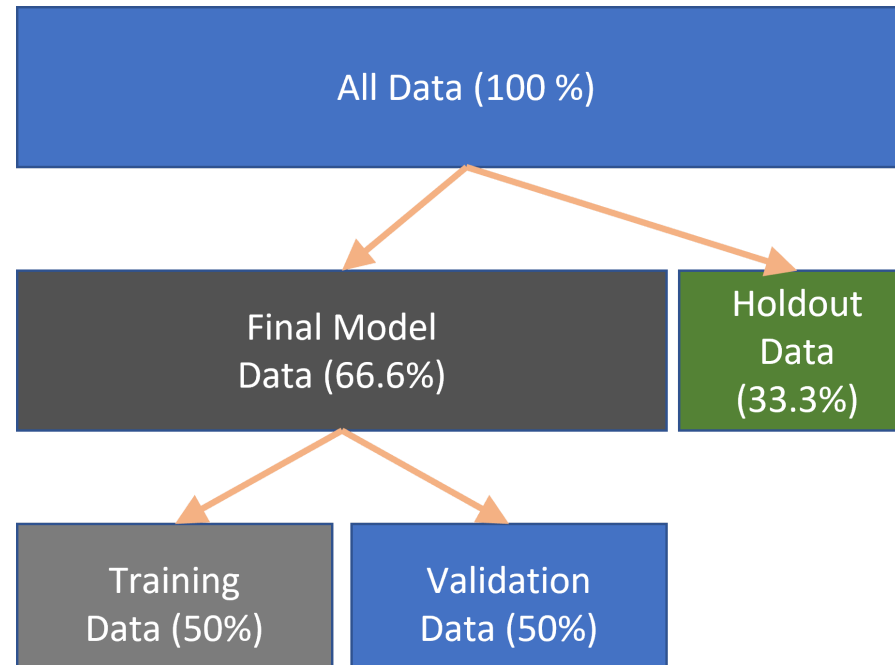
Following the diagram below, create the following variables:

`final_model` - which will hold two-thirds of the original data

`holdout` - which will hold one-third of the original data

`train` - which will hold 1/2 of the `final_model` data

validation - which will hold the other 1/2 of the final_model data



```
In [106]: 1 final_model, holdout = train_test_split(prediction_universe,
2                                             train_size = .666,
3                                             shuffle=True
4 )
5 train, validation = train_test_split(final_model,
6                                     train_size = .5,
7                                     shuffle=True
8 )
```

Part 2 (0.5 points)

Create the following Data Frames and Series:

Predictor DataFrames

- X_final_model - this will hold the predictors from final_model
- X_holdout - this will hold the predictors from holdout
- X_train - this will hold the predictors from train

- X_validation - this will hold predictors from validation

Outcome Series

- y_any_final_model - this will hold outcome_any_rearrest from final_model
- y_felony_final_model - this will hold outcome_felony_rearrest from final_model
- y_any_train - this will hold outcome_any_rearrest from train
- y_felony_train - this will hold outcome_felony_rearrest from train

Use the following predictors:

- arrests__age
- arrests__total_2016
- arrests__total_2017
- arrests__total_2018
- charges__num_B__last_two_years
- arrests__weapon_used

```
In [107]: 1 # This list will be helpful in creating the four predictor DataFrames
2 features = ['arrests__age', 'arrests__total_2016', 'arrests__total_2017',
3            'arrests__total_2018', 'charges__num_B__last_two_years',
4            'arrests__weapon_used']
5
6
7
8 X_final_model = final_model[features]
9 X_holdout = holdout[features]
10 X_train = train[features]
11 X_validation = validation[features]
12
13 y_any_final_model = final_model['outcome__any']
14 y_felony_final_model = final_model['outcome__felony']
15 y_any_train = train['outcome__any']
16 y_felony_train = train['outcome__felony']
17
```

Predict Any Rearrest

We will compare the performance of two models:

- A decision tree with max_depth of 3
- A random forest model with a max_depth of 3

```
In [108]: 1 dt_2 = dt(max_depth=2)
          2 rf_2 = rf(max_depth=2)
```

Model training (0.5 points)

Train the two models to predict any rearrest

```
In [109]: 1 x_test = pd.get_dummies(data=X__train, columns=['charges__num_B__last_two_years'])
          2
          3 dt_2.fit(X=x_test, y=y__any__train)
          4 dt_2.predict(X=x_test)
          5
          6 rf_2.fit(X=x_test, y=y__any__train)
          7 rf_2.predict(X=x_test)
```

```
Out[109]: ▼      DecisionTreeClassifier
           DecisionTreeClassifier(max_depth=2)
```

```
Out[109]: array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

```
Out[109]: ▼      RandomForestClassifier
           RandomForestClassifier(max_depth=2)
```

```
Out[109]: array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

Predict for validation (0.5 points)

Create the following two columns in `validation` that will hold the predicted probabilities from the models when predicting for the validation set:

1. `pred__any__dt2`

2. pred__any__rf2

```
In [110]: 1 X_train_dummies = pd.get_dummies(data=X_train, columns=['charges__num_B__last_two_years'])
2 X_validation_dummies = pd.get_dummies(data=X_validation, columns=['charges__num_B__last_two_years'])
3 missing_cols = set(X_train_dummies.columns) - set(X_validation_dummies.columns)
4 for col in missing_cols:
5     X_validation_dummies[col] = 0
6 extra_cols = set(X_validation_dummies.columns) - set(X_train_dummies.columns)
7 X_validation_dummies.drop(columns=extra_cols, inplace=True)
8
9 dt_2.fit(X=X_validation_dummies, y= validation['outcome__any'])
10 dt2_pred = dt_2.predict(X=X_validation_dummies)
11 rf_2.fit(X=X_validation_dummies, y= validation['outcome__any'])
12 rf2_pred = rf_2.predict(X=X_validation_dummies)
13
14 validation['pred__any__dt2'] = dt2_pred
15 validation['pred__any__rf2'] = rf2_pred
16
```

Out[110]:

DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)

Out[110]:

RandomForestClassifier
RandomForestClassifier(max_depth=2)

Convert predicted probabilities to predicted outcomes (0.5 points)

Create the following two columns in `validation` that will hold the predicted outcomes from the models when predicting for the validation set:

1. `yhat__any__dt2`
2. `yhat__any__rf2`

(Make sure to use `threshold__any`)

```
In [111]: 1 validation['yhat__any__dt2'] = validation.pred__any__dt2.rank(ascending=False, method='first') <= thr
          2 validation['yhat__any__rf2'] = validation.pred__any__rf2.rank(ascending=False, method='first') <= thr
          3
```

Compute PPV (0.5 points)

Use the `yhat` and `outcome` columns to compute PPV (Positive Predictive Value or precision) for both models

```
In [112]: 1 PPV_dt2 = validation[validation['yhat__any__dt2']==1]['outcome__any'].mean()
          2 PPV_rf2 = validation[validation['yhat__any__rf2']==1]['outcome__any'].mean()
          3
          4 PPV_dt2
          5 PPV_rf2
```

```
Out[112]: 0.182
```

```
Out[112]: 0.14
```

Predict for holdout and convert to yhat (0.5 points)

Which model had the higher PPV? Use that model (or select either if they have identical performance) to:

- Create a column called `pred__any` in `holdout` which holds predicted probabilities of any rearrest for the holdout set
- Create a column called `yhat__any` in `holdout` which holds the predicted outcomes for the holdout set

```

In [113]: 1 #The decision tree model has the higher PPV
          2
          3 x_hold = pd.get_dummies(data=X__holdout, columns=['charges__num_B__last_two_years'])
          4 x_hold
          5 dt_2.fit(X=x_hold, y=holdout['outcome__any'])
          6 dt2_pred_hold = dt_2.predict(X=x_hold)
          7
          8
          9 holdout['pred__any'] = dt2_pred_hold
         10 holdout['yhat__any'] = holdout.pred__any.rank(ascending=False, method='first') <= threshold__any
         11 holdout

```

Out[113]:

	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	arrests__weapon_used	charges__num_B__last_two_
1064	41.0	0.0	0.0	1.0	1	
23	24.0	0.0	0.0	1.0	1	
5611	29.0	0.0	0.0	1.0	1	
2222	36.0	0.0	0.0	1.0	1	
4129	32.0	0.0	0.0	1.0	1	
...	...	...	...	...	...	...
517	34.0	0.0	0.0	1.0	1	
3761	50.0	0.0	0.0	1.0	1	
5461	31.0	0.0	0.0	1.0	1	
5076	80.0	0.0	0.0	1.0	1	
32	27.0	0.0	0.0	1.0	1	

2593 rows × 20 columns



Out[113]:

```

DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)

```

Out[113]:

	ArresteeID	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	group	outcome__any	outcome__felo
1064	101672282.0	41.0	0.0	0.0	1.0	0.0	0	
23	101637640.0	24.0	0.0	0.0	1.0	0.0	0	
5611	101864649.0	29.0	0.0	0.0	1.0	0.0	0	
2222	101806205.0	36.0	0.0	0.0	1.0	0.0	0	
4129	100262016.0	32.0	0.0	0.0	1.0	1.0	0	
...	...	...	...	...	...	...	...	...
517	101747375.0	34.0	0.0	0.0	1.0	0.0	0	
3761	101616692.0	50.0	0.0	0.0	1.0	1.0	0	
5461	101733478.0	31.0	0.0	0.0	1.0	0.0	0	
5076	101490875.0	80.0	0.0	0.0	1.0	1.0	0	
32	101740079.0	27.0	0.0	0.0	1.0	0.0	0	

2593 rows × 13 columns



Repeat the steps above for the felony prediction (2.5 points)

1. Train model
2. Predict for validation
3. Create y-hat for validation
4. Compute PPV
5. Predict for holdout and convert yhat

Make sure the column names are the same as above, but change `any` to `felony`

(If your two models give the same PPV, just choose either when predicting for holdout)

```
In [114]: ▶ 1 dt_2 = dt(max_depth=2)
2 rf_2 = rf(max_depth=2)
3
4 x_test = pd.get_dummies(data=X__train, columns=['charges__num_B__last_two_years'])
5
6 dt_2.fit(X=x_test, y=y__felony__train)
7 dt_2.predict(X=x_test)
8
9 rf_2.fit(X=x_test, y=y__felony__train)
10 rf_2.predict(X=x_test)
```

```
Out[114]: ▼      DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)
```

```
Out[114]: array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

```
Out[114]: ▼      RandomForestClassifier
RandomForestClassifier(max_depth=2)
```

```
Out[114]: array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

```

In [115]: ► 1 X_train_dummies = pd.get_dummies(data=X_train, columns=['charges__num_B__last_two_years'])
2 X_validation_dummies = pd.get_dummies(data=X_validation, columns=['charges__num_B__last_two_years'])
3 missing_cols = set(X_train_dummies.columns) - set(X_validation_dummies.columns)
4 for col in missing_cols:
5     X_validation_dummies[col] = 0
6 extra_cols = set(X_validation_dummies.columns) - set(X_train_dummies.columns)
7 X_validation_dummies.drop(columns=extra_cols, inplace=True)
8
9 dt_2.fit(X=X_validation_dummies, y= validation['outcome__felony'])
10 dt2_pred = dt_2.predict(X=X_validation_dummies)
11 rf_2.fit(X=X_validation_dummies, y= validation['outcome__felony'])
12 rf2_pred = rf_2.predict(X=X_validation_dummies)
13
14 validation['pred__felony__dt2'] = dt2_pred
15 validation['pred__felony__rf2'] = rf2_pred
16
17 validation['yhat__felony__dt2'] = validation.pred__any__dt2.rank(ascending=False, method='first') <=
18 validation['yhat__felony__rf2'] = validation.pred__any__rf2.rank(ascending=False, method='first') <=
19

```

Out[115]:

```

▼      DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)

```

Out[115]:

```

▼      RandomForestClassifier
RandomForestClassifier(max_depth=2)

```

```
In [116]: ▶ 1 PPV_dt2 = validation[validation['yhat__felony__dt2']==1]['outcome__felony'].mean()
2 PPV_rf2 = validation[validation['yhat__felony__rf2']==1]['outcome__felony'].mean()
3 PPV_dt2
4 PPV_rf2
5
6 x_hold = pd.get_dummies(data=X__holdout, columns=['charges__num_B__last_two_years'])
7
8 dt_2.fit(X=x_hold, y=holdout['outcome__felony'])
9 dt2_pred_hold = dt_2.predict(X=x_hold)
10
11
12 holdout['pred__felony'] = dt2_pred_hold
13 holdout['yhat__felony'] = holdout.pred__felony.rank(ascending=False, method='first') <= threshold_fe
14
```

Out[116]: 0.11

Out[116]: 0.05

Out[116]:

▼	DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)	

Out[116]: 0

Calculate Impact

Online Therapy Program

Number of arrests before the intervention (1 point)

Created the following two variables:

- `flagged__nonfelony__any_model` - The number of nonfelony rearrests among those predicted to be at high risk of any rearrest in the absence of the intervention

```
In [117]: 1 flagged__nonfelony__any_model = holdout[(holdout['outcome__nonfelony']==1) & (holdout['yhat__any']==1)
2 flagged__felony__any_model = holdout[(holdout['outcome__felony']==1) & (holdout['yhat__any']==1)].sha
3
4 flagged__nonfelony__any_model
5 flagged__felony__any_model
```

Out[117]: 62

Out[117]: 20

Effect of intervention (1 point)

Using information about the treatment effect of the intervention, create the following two variables:

- `change__nonfelony__any_model` - The change in the number of nonfelony rearrests among those predicted to be at high risk of any rearrest after the intervention
- `change__felony__any_model` - The change in the number of felony rearrests among those predicted to be at high risk of any rearrest after the intervention

```
In [118]: 1 change__nonfelony__any_model = flagged__nonfelony__any_model*treatment_effect__online_therapy
2 change__felony__any_model = (flagged__felony__any_model*treatment_effect__online_therapy)
3
4 change__nonfelony__any_model = -change__nonfelony__any_model
5 change__felony__any_model = -change__felony__any_model
6
7 change__nonfelony__any_model
8 change__felony__any_model
```

Out[118]: -12.4

Out[118]: -4.0

Converting into dollar values (1 point)

Assume that:

- a nonfelony crime costs society \$5,000 per nonfelony crime
- a felony crime costs society \$100,000 per felony crime

Compute: benefit__online_therapy : the decline in the social cost of crime as a result of the online therapy intervention.

```
In [119]: 1 cost_nonfelony = 5000  
          2 cost_felony = 100000
```

```
In [120]: 1 benefit_nonfelony = change__nonfelony__any_model * cost_nonfelony  
          2 benefit_felony = change__felony__any_model * cost_felony  
          3 benefit__online_therapy = benefit_nonfelony + benefit_felony  
          4 round(benefit__online_therapy)
```

Out[120]: -462000

Calculate the social cost of Online Therapy (1 point)

Assume that each person who is offered online therapy but who would not have been re-arrested (i.e. a False Positive) suffers a social cost \$1,000.

1. Calculate num_FP__online_therapy : the number of False Positives among those at high risk of any rearrest. (A false positive here is someone who is predicted to be at high risk of any rearrest but is not re-arrested for a crime.)
2. Calculate social_cost_online_therapy : the social cost of Online Therapy
3. Compute total_benefit_online_therapy : the combined cost/benefit of the program (i.e. combining results from previous question with this one)

```
In [121]: 1 cost_online_therapy=1000
```

```
In [122]: 1 num_FP__online_therapy = holdout[(holdout['outcome__any']==0) & (holdout['yhat__any']==1)].shape[0]  
          2 num_FP__online_therapy  
          3  
          4 social_cost_online_therapy = num_FP__online_therapy*cost_online_therapy  
          5 social_cost_online_therapy
```

Out[122]: 418

Out[122]: 418000

```
In [123]: 1 total_benefit_online_therapy = social_cost_online_therapy + benefit__online_therapy
          2 round(total_benefit_online_therapy)
```

Out[123]: -44000

Detention

Number of arrests before the intervention (1 point)

Created the following two variables:

- `flagged__nonfelony__felony_model` - The number of nonfelony rearrests among those predicted to be at high risk of felony rearrest before the intervention
- `flagged__felony__felony_model` - The number of felony rearrests among those predicted to be at high risk of felony rearrest before the intervention

```
In [124]: 1 flagged__nonfelony__felony_model = holdout[(holdout['outcome__nonfelony']==1) & (holdout['yhat__felon
          2 flagged__felony__felony_model = holdout[(holdout['outcome__felony']==1) & (holdout['yhat__felony']==1
          3
          4 flagged__nonfelony__felony_model
          5 flagged__felony__felony_model
```

Out[124]: 17

Out[124]: 4

Effect of intervention (1 point)

Using information about the treatment effect of the intervention, create the following two variables:

- `change__nonfelony__felony_model` - The change in the number of nonfelony rearrests among those predicted to be at high risk of felony rearrest after the intervention
- `change__felony__felony_model` - The change in the number of felony rearrests among those predicted to be at high risk of felony rearrest after the intervention

```
In [134]: 1 change__nonfelony__felony_model = flagged__nonfelony__felony_model*treatment_effect__detention
          2 change__felony__felony_model = flagged__felony__felony_model*treatment_effect__detention
          3
          4 change__nonfelony__felony_model = -change__nonfelony__felony_model
          5 change__felony__felony_model = -change__felony__felony_model
          6
          7 change__nonfelony__felony_model
          8 change__felony__felony_model
```

Out[134]: -11.899999999999999

Out[134]: -2.8

Converting into dollar values (1 point)

As above, assume that:

- a nonfelony crime costs society \$5,000 per nonfelony crime
- a felony crime costs society \$100,000 per felony crime

Compute `benefit_detention` : the decline in the social cost of crime as a result of detention

```
In [135]: 1 benefit_nonfelony = change__nonfelony__felony_model * cost_nonfelony
          2 benefit_felony = change__felony__felony_model * cost_felony
          3 benefit_detention = benefit_nonfelony + benefit_felony
          4 round(benefit_detention)
```

Out[135]: -339500

Calculate the social cost of detention (1 point)

Assume that each person falsely detained suffers a social cost \$30,000.

1. Calculate `num_FP_detention` the number of False Positives among those at high risk of felony rearrest. (A false positive here is someone who is predicted to be at high risk of felony rearrest but is not re-arrested for a felony crime.)
2. Compute `social_cost_detention` : the social cost of detention

```
In [127]: 1 cost_detention = 30000
```

```
In [139]: 1 num_FP__detention = holdout[(holdout['outcome__felony']==0) & (holdout['yhat__felony']==1)].shape[0]
2 num_FP__detention
3
4 social_cost_online_therapy = num_FP__detention*cost_detention
5 social_cost_online_therapy
```

```
Out[139]: 96
```

```
Out[139]: 2880000
```

```
In [137]: 1 total_benefit_detention = social_cost_online_therapy + benefit_detention
2 round(total_benefit_detention)
```

```
Out[137]: 2540500
```

Fairness

PPV Balance (1 point)

1. In your own words, describe why we would want an algorithm to achieve balance in PPV? (Think about NorthPointe's justification re: Compas algorithm)
2. Compute the PPV for those at high risk of any-rearrest (when evaluating on `outcome__any`) separately for the two groups
3. Compute the PPV for those at high risk of felony-rearrest (when evaluating on `outcome__felony`) separately for the race groups

```

In [130]: 1 # A high PPV means fewer false positives, meaning fewer instances where someone is wrongly labeled as
          2 holdout0 = holdout[(holdout['group'] == 0)]
          3 holdout1 = holdout[(holdout['group'] == 1)]
          4
          5 ppv_g0_any = holdout0[holdout0['yhat__any']==1]['outcome__any'].mean()
          6 ppv_g1_any = holdout1[holdout1['yhat__any']==1]['outcome__any'].mean()
          7
          8 ppv_g0_fel = holdout0[holdout0['yhat__felony']==1]['outcome__felony'].mean()
          9 ppv_g1_fel = holdout1[holdout1['yhat__felony']==1]['outcome__felony'].mean()
         10
         11 ppv_g0_any
         12 ppv_g1_any
         13 ppv_g0_fel
         14 ppv_g1_fel

```

Out[130]: 0.1274131274131274

Out[130]: 0.2033195020746888

Out[130]: 0.018518518518518517

Out[130]: 0.06521739130434782

FPR Balance (1 point)

1. In your own words, describe why we would want an algorithm to achieve balance in False Positive Rates? (Think about Pro Publica's argument)
2. Compute the FPR for those at high risk of any-rearrest (when evaluating on `outcome__any`) separately for the two groups
3. Compute the FPR for those at high risk of felony-rearrest (when evaluating on `outcome__felony`) separately for the two groups

```
In [131]: 1 # Falsely Labelling someone as high-risk because of their race or other inherent factors is unjust an
2 holdout0 = holdout[(holdout['group'] == 0)]
3 holdout1 = holdout[(holdout['group'] == 1)]
4
5 fpr_g0_any = holdout0[(holdout0["outcome__any"] == 0) & (holdout0["yhat__any"] == 1)].shape[0]/(holdo
6 fpr_g1_any = holdout1[(holdout1["outcome__any"] == 0) & (holdout1["yhat__any"] == 1)].shape[0]/(holdo
7
8 fpr_g0_fel = holdout0[(holdout0["outcome__felony"] == 0) & (holdout0["yhat__felony"] == 1)].shape[0]/
9 fpr_g1_fel = holdout1[(holdout1["outcome__felony"] == 0) & (holdout1["yhat__felony"] == 1)].shape[0]/
10
11 fpr_g0_any
12 fpr_g1_any
13 fpr_g0_fel
14 fpr_g1_fel
```

Out[131]: 0.1628242074927954

Out[131]: 0.1596009975062344

Out[131]: 0.03818443804034582

Out[131]: 0.03574397339983375

FNR Balance (1 point)

1. In your own words, describe why we would want an algorithm to achieve balance in False Negative Rates? (Think about Pro Publica's argument)
2. Compute the FNR for those at high risk of any-rearrest (when evaluating on `outcome__any`) separately for the two groups
3. Compute the FNR for those at high risk of felony-rearrest (when evaluating on `outcome__felony`) separately for the two groups

```
In [132]: 1 # If an algorithm consistently misses real threats, people might lose trust in its ability to correct
2 holdout0 = holdout[(holdout['group'] == 0)]
3 holdout1 = holdout[(holdout['group'] == 1)]
4
5 fnr_g0_any = holdout0[(holdout0["outcome__any"] == 1) & (holdout0["yhat__any"] == 0)].shape[0]/(holdout0["outcome__any"] == 1).shape[0]
6 fnr_g1_any = holdout1[(holdout1["outcome__any"] == 1) & (holdout1["yhat__any"] == 0)].shape[0]/(holdout1["outcome__any"] == 1).shape[0]
7
8 fnr_g0_fel = holdout0[(holdout0["outcome__felony"] == 1) & (holdout0["yhat__felony"] == 0)].shape[0]/(holdout0["outcome__felony"] == 1).shape[0]
9 fnr_g1_fel = holdout1[(holdout1["outcome__felony"] == 1) & (holdout1["yhat__felony"] == 0)].shape[0]/(holdout1["outcome__felony"] == 1).shape[0]
10
11 fnr_g0_any
12 fnr_g1_any
13 fnr_g0_fel
14 fnr_g1_fel
```

Out[132]: 0.11311239193083573

Out[132]: 0.1371571072319202

Out[132]: 0.028097982708933718

Out[132]: 0.04987531172069826

Make your decision (3 points)

You have generated the following:

- The social benefit of online therapy
- The social cost of online therapy
- The social benefit of detention
- The social cost of detention
- The three disparities metrics for both algorithms

Recall that the policy objective of your community is to reduce crime by intervening with people who are high risk. One intervention targets violent offenses and involves detaining the high risk. The other intervention targets all offenses and involves offering individuals online therapy.

Use the information you have generated above as well as what we learned in class to discuss the pros and cons of choosing one algorithm over the other in terms of accuracy, fairness (who bears the cost of the algorithm's mistakes), and ability to meet the policy goal

```
In [ ]: ▶ 1 #The online therapy algorithm would be the way to go, it gives back more to society and also has the  
2 #attending to a larger population. Also the cost of felonys significantly outweighs the cost of deten  
3 #the algorithm for detecting false positives in the online therapy program is more accurate meaning L  
4 #positives being added to the cost of felony and a lot more added to the cost of detention.  
5
```

Extract Credit I (1 point)

How long did it take you to do this final? (Your answer will have no bearing on your grade)

```
In [ ]: ▶ 1 #Probably 15 total hours...
```

Extra Credit II (5 total points)

- Using the `train` dataset, run a new random forest model with GridSearchCV that test tree depths of 2, 4, and 10 and that predicts felony rearrest. This time we are ADDITIONALLY using the `group` column as a predictor. (1 point)
- Predict for the validation set (1 point)
- Compute the three fairness metrics (2 points)
- Did the algorithm become more or less fair after we included race? (1 point)

```

In [133]: ► 1 features2 = ['arrests__age', 'arrests__total_2016', 'arrests__total_2017',
2             'arrests__total_2018', 'charges__num_B__last_two_years',
3             'arrests__weapon_used', 'group']
4
5
6 param_grid_rf = {'max_depth' : [2, 4, 10]}
7 rf_3 = rf()
8 gs_rf = GridSearchCV(estimator=rf_3,
9                       param_grid=param_grid_rf,
10                      cv=5)
11 x_train = train[features2]
12 x_train
13 x_train = pd.get_dummies(data=x_train, columns=['charges__num_B__last_two_years', 'group'])
14 gs_rf.fit(X=x_train, y=train['outcome__felony'])
15 gs_rf_pred = gs_rf.predict(X=x_train)
16
17 train['pred__felony'] = gs_rf_pred
18 train['yhat__felony'] = train.pred__felony.rank(ascending=False, method='first') <= threshold__felony
19
20 train0 = train[(train['group'] == 0)]
21 train1 = train[(train['group'] == 1)]
22
23 ppv_g0_fel = train0[train0['yhat__felony']==1]['outcome__felony'].mean()
24 ppv_g1_fel = train1[train1['yhat__felony']==1]['outcome__felony'].mean()
25
26 fpr_g0_fel = train0[(train0["outcome__felony"] == 0) & (train0["yhat__felony"] == 1)].shape[0]/(train
27 fpr_g1_fel = train1[(train1["outcome__felony"] == 0) & (train1["yhat__felony"] == 1)].shape[0]/(train
28
29 fnr_g0_fel = train0[(train0["outcome__felony"] == 1) & (train0["yhat__felony"] == 0)].shape[0]/(train
30 fnr_g1_fel = train1[(train1["outcome__felony"] == 1) & (train1["yhat__felony"] == 0)].shape[0]/(train
31
32 ppv_g0_fel
33 ppv_g1_fel
34 fpr_g0_fel
35 fpr_g1_fel
36 fnr_g0_fel
37 fnr_g1_fel
38
39 #It is more fair when including group as a feature because the PPV is higher and the FPR and FNR are

```

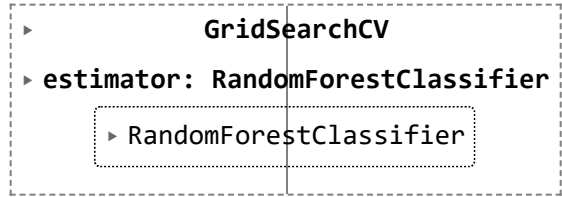
Out[133]:

	arrests__age	arrests__total_2016	arrests__total_2017	arrests__total_2018	charges__num_B__last_two_years	arrests__weapon
454	71.0	0.0	0.0	1.0	1.0	
1349	24.0	0.0	0.0	1.0	1.0	
4335	31.0	0.0	0.0	2.0	2.0	
5486	51.0	0.0	0.0	1.0	1.0	
5051	49.0	0.0	0.0	1.0	1.0	
...	...	...	...	...	...	...
3119	43.0	1.0	0.0	1.0	1.0	
4488	70.0	0.0	0.0	1.0	1.0	
3124	36.0	0.0	0.0	2.0	2.0	
1493	62.0	0.0	0.0	1.0	1.0	
4394	23.0	0.0	0.0	1.0	1.0	

2585 rows × 7 columns



Out[133]:



Out[133]: 0.2619047619047619

Out[133]: 0.3103448275862069

Out[133]: 0.022644265887509132

Out[133]: 0.03289473684210526

Out[133]: 0.02191380569758948

Out[133]: 0.04276315789473684

